

三角形

手玩一下就能发现多次翻转可以转化为旋转 + 左右翻转 + 旋转的情况。

只要枚举每种情况即可。

左右翻转是容易的。

旋转的操作可以模拟一下操作，就发现原 (i, j) 的数会转到 $(n - i + j, n - i + 1)$ 。

然后就做完了。

子段异或

题意：在一个 01 串中找出两个子串作为二进制下的两个数，使得它们的异或结果最大。

考虑贪心：

首先想到首位为 1 的最长的串，如果能保留下这位 1，那么它的结果一定比所有其他不取最长串方案更大。

容易发现除了总长为 1 或全是 0 的串的极端情况外一定能保留下这个最大的 1，于是我们找到第一个出现 1 的位置 fst ，第一个子串取 $fs(fst, n)$ 一定能保证答案最大。

0000000110100.....100
fst fs(fst,n) 洛谷

接下来考虑第二个串的取法。对于 fst 以及其右边相邻的一整块 1，显然保留它们更好。考虑到 fst 之后第一个 0 的位置 z ，如果第二个串能让 z 位置上的 0 异或后变成 1，结果的答案一定比 z 上异或后不是 1 的方案更优。

因为 z 的左边有着和 fst 相邻的一整块 1，也就是一定有对齐之后最高位的 1 正好在 z 上的串存在，只要考虑在它们中取左边的哪个 1 作为开头即可。

再考虑 z 右边最长的相邻的一整块 0，记它们的右边界为 r ，在理想情况下我们让 $z \sim r$ 中的 0 都能变成 1，同时利用 $z \sim r$ 这些 0 让 r 右边的第一个 1 不受影响。可以发现这种取法是一致的并且一定最优，只要让 $\max(fst, z - (r - z + 1))$ 作为第一个串的开头就行了。

res 11111111111111.....
f2 1110001111.....0
f1 0011111110001111.....0010
fst z-(r-z+1) z r 洛谷

最后考虑一些边界情况：如果全是 1 那么输出 $n - 1$ 个 1 和一个 0；如果全是 0 输出 0；如果 fst 右边没有 0 那么输出长为 $n - fst + 1$ 个 1。

code:

```
1 #include<bits/stdc++.h>
2 using namespace std;
```

```

3  const int N=1e7+10;
4  int T,n;
5  string s;
6  bool a[N];
7  signed main(){
8      freopen("xor.in","r",stdin);
9      freopen("xor.out","w",stdout);
10     cin>>T;
11     while(T--){
12         cin>>n>>s;
13         for(int i=1;i<=n;i++){
14             a[i]=(s[i-1]-'0');
15         }
16         int fst=0,scd=0,z=0;
17         for(int i=1;i<=n;i++){
18             if(a[i]){
19                 if(!fst){
20                     fst=i;
21                 }
22             }else{
23                 if(fst){
24                     z=i;
25                     int r=z;
26                     while(r+1<=n&&(!a[r+1])){
27                         r++;
28                     }
29                     scd=max(fst,z-(r-z+1));
30                     break;
31                 }
32             }
33         }
34         if(fst==1&&(!z)){
35             for(int i=1;i<=n-1;i++){
36                 putchar('1');
37             }
38             putchar('0');
39             cout<<"\n";
40             continue;
41         }
42         if(!fst){
43             cout<<"0\n";
44             continue;
45         }
46         if(!z){
47             for(int i=fst;i<=n;i++){
48                 putchar('1');
49             }
50             cout<<"\n";
51             continue;
52         }
53         for(int i=fst;i<=n;i++){
54             if(i<z){
55                 putchar(a[i]+'0');
56             }else{
57                 putchar((a[i]^a[scd+i-z])+'0');
58             }

```

```

59     }
60     cout<<"\n";
61 }
62 return 0;
63 }
64

```

三等分

考虑将每个三元组按照第一个数字排序。我们称三元组中第一个数字为 i 的三元组为 i 类三元组

那么就等价于用若干个三元组，使得所有三元组中的每种数字 i 的出现次数恰好等于原序列中对应数字的出现次数 a_i

那么考虑 $f_{i,xy}$ 表示考虑了 1 到 i 类的三元组， $i-1$ 类中形如 $(a, a+1, a+2)$ 的有 x 个， i 类中形如 $(a, a+1, a+2)$ 的有 y 个时，的方案数

考虑转移：从 $f_{i-1,x,y}$ 转移到 $f_{i,y,?}$

此时数字 i 还有 $las1 = a[i] - x - y$ 个没有用， $i+1$ 还有 $las2 = a[i+1] - y$ 个没有用， $i+2$ 还有 $las3 = a[i+2]$ 个没有用

我们枚举有 k 个 i 类三元组是形如 $(x, x+1, x+2)$ 的，剩下的 $las1 - k$ 个需要用 (x, x, x) 类型的填

所以必须要满足 $las1 - k$ 是 3 的倍数，而且 $k \leq \min(las1, las2, las3)$

所以就应该是 $f_{i-1,x,y}$ 转移到 $f_{i,y,k}$

可以通过前缀和优化（严格来说是差分）来把转移变成 $O(1)$

分析复杂度：

看似这个是 n^3 的，但是 dp 后两个维度必须要 $\leq a[i]$

所以实际上复杂度是 $\sum_{i=1}^m a_i^2 \leq n^2$

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  typedef long long ll;
4  inline ll read() {
5      ll x = 0, f = 1;
6      char ch = getchar();
7      while (ch < '0' || ch > '9') {
8          if (ch == '-') f = -1;
9          ch = getchar();
10     }
11     while (ch >= '0' && ch <= '9') {
12         x = (x << 1) + (x << 3) + (ch ^ 48);
13         ch = getchar();
14     } return x * f;
15 }
16 const int N = 5e3 + 10, mod = 1e9 + 7;
17 void ad(int &x, ll y) {
18     ll nw = x + y;
19     while (nw >= mod) nw -= mod;
20     while (nw <= -mod) nw += mod;

```

```

21     x = nw;
22 }
23 int f[2][N][N], lim[2][N][3]; int o, t; int n, m; int a[N];
24 int main() {
25     freopen("three.in", "r", stdin);
26     freopen("three.out", "w", stdout);
27     memset(lim, -1, sizeof lim);
28     n = read(), m = read();
29     for (int i = 1; i <= n; i++) {
30         int nw = read(); a[nw]++;
31     } o = 0; t = 1;
32     f[0][0][0] = 1; f[0][0][3] = -1; lim[0][0][0] = 3; //lim[i][x][s] 表示
f[i][x] 对应中最大的有值的 y 的位置, s表示起始余数是0/1/2
33     for (int i = 1; i <= m; i++) {
34         int up = (i >= 2) ? a[i-2] : 0;
35         for (int x = 0; x <= up; x++) {
36             for (int s = 0; s < 3; s++) {
37                 int &u = lim[o][x][s];
38                 if (u == -1) continue;
39                 for (int y = s; y <= u; y += 3) {
40                     if (y >= 3) {
41                         ad(f[o][x][y], f[o][x][y-3]); f[o][x][y-3] = 0;
42                     }
43                     int nw = f[o][x][y]; if (!nw) continue;
44                     int lw = min({a[i]-x-y, a[i+1]-y, a[i+2]});
45                     if (lw < 0) continue;
46                     int ns = (a[i]-x-y) % 3;
47                     if (ns > lw) continue;
48                     int to = (lw - ns) / 3;
49                     to = to * 3 + ns + 3;
50                     ad(f[t][y][ns], nw); ad(f[t][y][to], -nw);
51                     lim[t][y][ns] = max(lim[t][y][ns], to);
52                 } f[o][x][u] = 0;
53                 u = -1;
54             }
55         }
56         swap(o, t);
57     }
58     ll ans = f[o][0][0]; ans = (ans % mod + mod) % mod;
59     cout << ans;
60     return 0;
61 }
62

```

二叉树遍历

求 x 在遍历序列中的位置, 等价于求有多少个点在 x 前被遍历。

考虑一下哪些情况下 y 会比 x 早遍历。

分别用 $lson(x)$ 和 $rson(x)$ 表示 x 的左右子树, 用 $sub(x)$ 表示 x 的子树, $op_x \in \{-1, 0, 1\}$ 表示 x 的遍历方式, 不难发现只有以下几种情况:

- $y \in lson(x)$, $op_x = -1$ 。
- 存在 z , 使得 $y \in lson(z)$, $x \in rson(z)$ 。

- $x \in \text{sub}(y), op_y = -1$ 。
- $x \in \text{rson}(y), op_y = 0$ 。

考虑把这四类分开计算贡献。

首先，前两类可以直接预处理。

对于后两类，考虑按编号分块，处理每个 y 对 x 的贡献。

由于只有区间赋值操作，所以可以把所有块分为两类：第一类为整个块所有点的遍历方式相同的块，第二类为其它块。

分别考虑两类块对 x 的贡献。

可以发现，第一类块对 x 的贡献只需要知道这个块内有多少 y ，满足 y 是 x 的祖先，和有多少 y ，满足 x 在 y 的右子树内。

考虑枚举 y ，然后就相当于若干次子树加，直接差分前缀和即可 $O(n\sqrt{n})$ 。

设 f_x 为所有第二类块内的点对 x 的贡献之和。

如果只有单点修改，可以发现修改一个点会对这个点的子树或右子树的 f_x 产生影响。

由于只存在区间赋值操作，所以每次操作最多新增

2 个二类块，在修改的时候如果遇到二类块就直接遍历这个块，然后把这个块变成一类块。

这样总共会有 $O(n\sqrt{n})$ 次单点修改， $O(n)$ 次查询，写个 $O(1) - O(\sqrt{n})$ 的分块平衡一下，总复杂度 $O(n\sqrt{n})$ 。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  const int N=1e5+5,T=350,M=N/T+5;
4  int n,m,son[N]
    [2],idx,dfn[N],em[N],eds[N],dy[N],siz[N],bh[N],st[N],ed[N],smk[N],smz[N],g1
    [M][N],g2[M][N],mk[N];
5  void dfs(int x){
6      siz[x]=1;
7      dfn[x]++;idx;
8      if(son[x][0])dfs(son[x][0]),siz[x]+=siz[son[x][0]];
9      em[x]=idx;
10     if(son[x][1])dfs(son[x][1]),siz[x]+=siz[son[x][1]];
11     eds[x]=idx;
12 }
13 void add(int x,int sm){
14     smk[bh[x]]+=sm;
15     smz[x]+=sm;
16 }
17 void add(int l,int r,int sm){
18     add(l,sm);
19     add(r+1,-sm);
20 }
21 int gets(int x){
22     int res=0;
23     for(int i=1;i<bh[x];i++)res+=smk[i];
24     for(int i=st[bh[x]];i<=x;i++)res+=smz[i];
25     return res;
26 }
27 int solve(int x){

```

```

28     int res=1+gets(dfn[x]);
29     int k=dy[x];
30     if(mk[bh[x]]!=2)k=mk[bh[x]];
31     if(k==0)res+=siz[son[x][0]];
32     if(k==1)res+=siz[son[x][0]]+siz[son[x][1]];
33     for(int i=1;i<=bh[n];i++){
34         if(mk[i]==2)continue;
35         if(mk[i]==-1)res+=g1[i][dfn[x]];
36         if(mk[i]==0)res+=g2[i][dfn[x]];
37     }
38     return res;
39 }
40 void upt(int x,int op){
41     if(dy[x]==-1)add(dfn[x]+1,eds[x],op);
42     if(dy[x]==0)add(em[x]+1,eds[x],op);
43 }
44 void Sets(int l,int r,int x){
45     int y=bh[l];
46     if(mk[y]!=2){
47         for(int i=st[y];i<=ed[y];i++)dy[i]=mk[y];
48         for(int i=l;i<=r;i++)dy[i]=x;
49         for(int i=st[y];i<=ed[y];i++)upt(i,1);
50         mk[y]=2;
51     }else{
52         for(int i=l;i<=r;i++){
53             upt(i,-1);
54             dy[i]=x;
55             upt(i,1);
56         }
57     }
58 }
59 void sets(int l,int r,int x){
60     if(bh[l]==bh[r]){
61         Sets(l,r,x);
62         return;
63     }
64     int ll=bh[l]+1,rr=bh[r]-1;
65     Sets(l,st[ll]-1,x);
66     Sets(ed[rr]+1,r,x);
67     for(int i=ll;i<=rr;i++){
68         if(mk[i]==2)for(int j=st[i];j<=ed[i];j++)upt(j,-1);
69         mk[i]=x;
70     }
71 }
72 signed main(){
73     ios::sync_with_stdio(0);
74     cin.tie(0);cout.tie(0);
75     cin>>n>>m;
76     for(int i=1;i<=n;i++){
77         cin>>son[i][0]>>son[i][1];
78         dy[i]=-1;
79         bh[i]=(i-1)/T+1;
80     }
81     for(int i=1;i<=bh[n];i++)mk[i]=2,st[i]=ed[i-1]+1,ed[i]=min(i*T,n);
82     dfs(1);
83     for(int i=1;i<=n;i++){

```

```

84     upt(i,1);
85     add(em[i]+1,eds[i],siz[son[i][0]]);
86     g1[bh[i]][dfn[i]+1]++;
87     g1[bh[i]][eds[i]+1]--;
88     g2[bh[i]][em[i]+1]++;
89     g2[bh[i]][eds[i]+1]--;
90 }
91 for(int i=1;i<=bh[n];i++){
92     for(int j=1;j<=n;j++)g1[i][j]+=g1[i][j-1],g2[i][j]+=g2[i][j-1];
93 }
94 while(m--){
95     int op,l,r,x;
96     cin>>op;
97     if(op==1){
98         cin>>l>>r>>x;
99         sets(l,r,x);
100     }else{
101         cin>>x;
102         cout<<solve(x)<<"\n";
103     }
104 }
105 }

```